Embedded Data Structure
(Lecture Note 6)

# Revisit C Programming (5)

2025. 3. 20

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

# Review of the Last Class

KU KONKUK UNIVERSITY

- **Memory allocation and structure**
  - **Memory allocation : key technique in embedded systems**
    - Remember the usage of "malloc" and "calloc"
    - Remember the memory allocation based on pointer of pointer
  - **Structure : a kind of cluster containing different modalities**
    - Very useful to handle data of different modalities in a unified way
    - Key technique in considering the data structure

**※ You need to be familiar with using memory allocation and structure !**

# Structure with Pointers (1/6)

- **Multiple structures with array-based representations**
  - **To store and use multiple sctructures**

```
struct ProgramClass{
    char name[100];
    int ID;
    int score;
    float attendance;
    float grade;
};
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| K | i | m | ₩0 | … | 1 | 90 | 4.3 |
| S | u | h | ₩0 | … | 2 | 88 | 4.3 |
| ⋮ | | | | | | | |
| Y | o | o | ₩0 | … | 9 | 85 | 4.0 |

```
#include   <stdio.h>

void main()
{
    struct ProgramClass s[45];

    strcpy(s[0].name, "Kim");
    s[0].ID = 20161111;
    s[0].score = 90;
    s[0].attendance = 1.0;
    s[0].grade = 4.3;

    strcpy(s[1].name, "Suh");
    …
}
```

# Structure with Pointers (2/6)

- **Two types of pointers related to structure**
  - **Pointer indicating the structure**

```
struct ProgramClass s = {"Kim", 20161111, 90, 1.0, 4.3};
struct ProgramClass *p;

p = &s;

printf("Name = %s, ID = %d, Score = %d, Attendance = %f, Grade = %f₩n",
        (*p).name, (*p).ID, (*p).score, (*p).attendance, (*p).grade);
```

**※ What is the difference btw. *p.name and (*p).name ?**

**※ When the pointer indicating the structure,**

**We can access the member using p->name, etc.**

**(That is, (*p).name is exactly same as p->name)**

**Very important**

# Structure with Pointers (3/6)

- **Two types of pointers related to structure – cont'd**
  - **Pointer as the structure member**

```
#include <stdio.h>

struct student{
    char name[100];
    int ID;
    int *score;
}

void main()
{
    int new_score = 95;
    s.score = &new_score;

    printf("%d₩n", [          ]);
}
```

```
#include <stdio.h>

struct student{
    char name[100];
    int ID;
    int **score;
}

void main()
{
    int new_score = 95;
    int *p = &new_score;
    s.score = [          ]

    printf("%d₩n", [          ]);
}
```

# Structure with Pointers (4/6)

- **Two types of pointers related to structure – cont'd**
  - **Structure including a pointer indicating itself**

```
struct student {
    char name[100];
    int ID;
    int score;
    struct student *next;
};
```

**We can link our structures using this internal pointer**

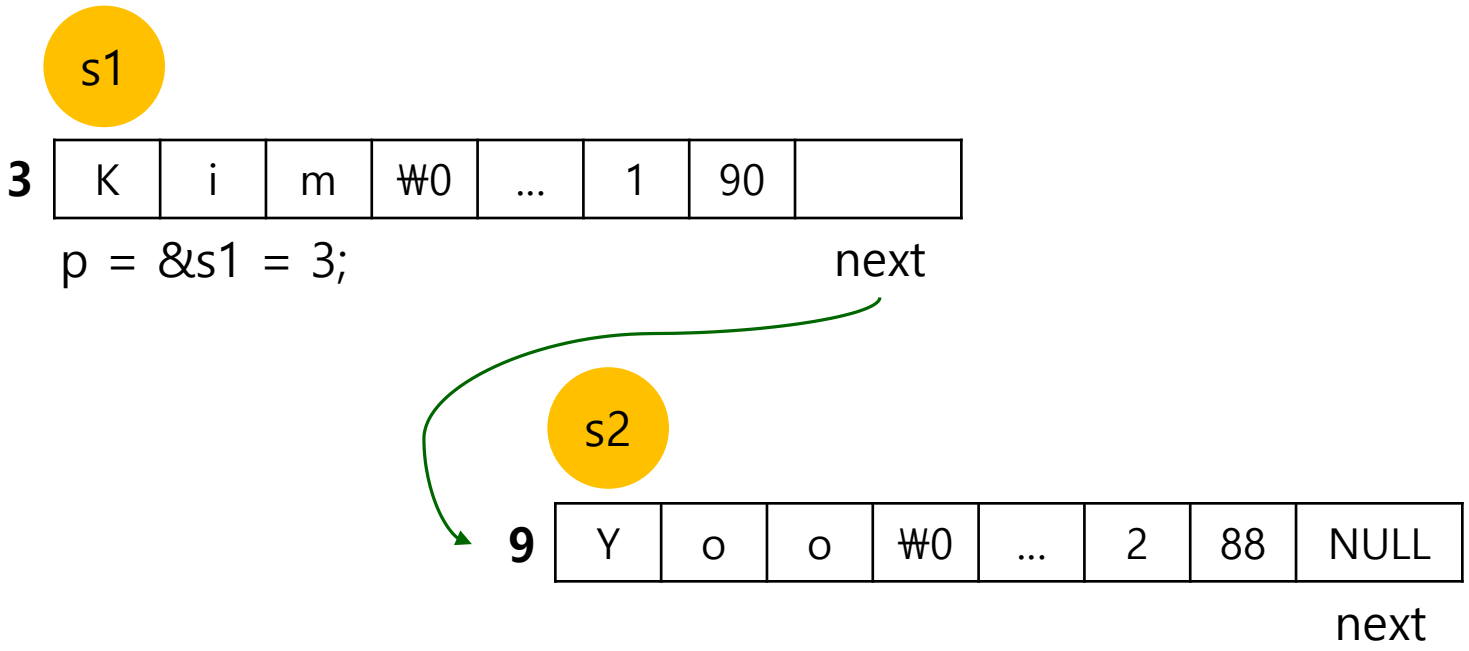
```
void main()
{
    struct student s1 = {"Kim", 1, 90, NULL};
    struct student s2 = {"Yoo", 2, 88, NULL};
    struct student *p = NULL;
    struct student *q = NULL;

    p = &s1;
    s1.next = &s2;
    s2.next = NULL;

    q = p;
    while(q != NULL){
        printf("%s,  %d,  %d₩n", q->....);
        [          ];
    }
}
```

# Structure with Pointers (5/6)

- **Two types of pointers related to structure – cont'd**
  - **Pictorial example for a simple linked list**




**Consider the procedure shown in the previous slide !**

# Structure with Pointers (6/6)

KU KONKUK UNIVERSITY

## ▪ Memory allocation in structures

- **Conduct memory allocation as follows :**

```
typedef struct _model{
    int val;
    float *color;
}model;

void main(){
    int k;
    model *object;

    [                    ]
    for(k = 0; k<5; k++){
    [                    ]
    }
}
```

**Very important example !**

**Five for object (memory alloc.)**

**Three for color (memory alloc.)**

(Think about two different representations)

# Structure and Function (1/3)

KU KONKUK UNIVERSITY

## ▪ Structure as the parameter or the return value

- **Structure as the parameter**

```c
#include <stdio.h>

struct node{
    int val;
    struct node *next;
};

int equal(struct node p, struct node q)
{
    if(p.val == q.val)
        return 1;
    else
        return 0;
}
```

```c
#include <stdio.h>

struct node{
    int val;
    struct node *next;
};

int equal(struct node *p, struct node *q)
{
    if( [                ] )
        return 1;
    else
        return 0;
}
```

➡ **How can we apply this function in the main function ?**

# Structure and Function (2/3)

KU KONKUK UNIVERSITY

## ▪ Structure as the parameter or the return value

- **Structure as the return value**

```
struct node function_A()
    strunct node s;
    s.val = 3;

    return s;
}

void main()
{
    struct node s;
    s = function_A();

    printf("%d₩n", s.val);
}
```

```
struct node *function_A()
    strunct node *s;
    s->val = 3;
    s->next = 9;

    return [          ];
}

void main()
{
    struct node [    ];
    [    ] = function_A();

    printf("%d₩n", [    ]);
}
```

# Structure and Function (3/3)

- **Example : insert a new node whose value is 'k' after …**
  - **Insert a new node after the current node**

```
struct node *insert_after(int k, struct node *t)
{
    Make your code !
}
```

```
void main()
{
    struct node *s1, *p;

    s1->val = 3;

}
```

# Practical Example

- **Think about following codes**
  - **If codes contain error, then fix them !**

```
struct book{
    char title[50];
    int pages;
};
book.pages = 512
```

```
struct book{
    char title[50] = "C";
    int pages = 300;
};
book b;
```

```
struct fraction{
    int val;
    int link;
}*p;

*p->val = 3;
*p->link = 9;
```

```
struct fraction{
    int val;
    int *link;
}*p;

int a = 9;
p->link = a;
```

# Summary

- **Structure and its applications**
  - **Structure with pointers**

    → It greatly helps to understand the data structure with memory alloc.

    (c.f.: very useful to understand "linked list")

  - **Structure and functions**

    - Structure as a parameter for function